

# Team Defense Game

The three channels between two defenders, and what each of them is called

---

The page states the question and then refuses the obvious version of it.

*“Two defenders, one network, one attacker working against them. The question isn’t which of them is better. It’s what passes between them.”*

website, Team Defense Game, opening

Three things pass. An approval, which decides whether a proposed action fires. A shared account, which every action spends from whether or not the two defenders agreed. And the outcome of an action the agent did not choose, which it stores anyway. This file names all three, in the game’s vocabulary and in the platform’s, and marks the four places where those two vocabularies say different things with the same word.

## Contents

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>1</b> | <b>Active</b>                               | <b>2</b>  |
| 1.1      | The game . . . . .                          | 2         |
| 1.2      | Approval . . . . .                          | 3         |
| 1.3      | The shared pool . . . . .                   | 4         |
| 1.4      | The agent’s memory . . . . .                | 5         |
| 1.5      | The board . . . . .                         | 7         |
| 1.6      | The evaluation . . . . .                    | 9         |
| <b>2</b> | <b>Working</b>                              | <b>10</b> |
| <b>3</b> | <b>Peripheral</b>                           | <b>11</b> |
| <b>4</b> | <b>The clashes, listed</b>                  | <b>12</b> |
| <b>5</b> | <b>What crosses into the shared lexicon</b> | <b>12</b> |
| <b>6</b> | <b>Sources</b>                              | <b>12</b> |

# 1 Active

## 1.1 The game

### Team Defense Game, TDG

*“Two defenders, one network, one attacker working against them.”*

website, Team Defense Game, opening

**Definition:** A two-defender cyber defense game on a simulated enterprise network, in which one defender is a person and the other is an autonomous agent, and the two are joined by an approval requirement, a shared cost account, and an agent memory that updates from both defenders’ outcomes.

**Distinguishes it from:** Three neighbours. From CybORG and the CAGE challenges, where the human sits outside the episode and reads the score. From the Interactive Defense Game, where a person plays the same scenario alone. From CHART, where the team structure is configured per condition rather than fixed. TDG is the case with exactly one human, exactly one agent, and one coordination pattern written into the game.

**Operational test:** Remove the agent and ask whether the game still runs. If it does, and nothing about the human’s options changes, the interdependence was decoration.

**In my project:** The book chapter dates the platform: TDG adapted CybORG for human-AI interaction studies, and in its initial form let a human defender take direct control of network defense actions. The three channels came later.

### Controlled-TDG

**Definition:** The build in which the coordination between the two defenders is itself a configurable object rather than a fixed rule. Two variants are kept side by side, one upstream and one adapted.

**Distinguishes it from:** The original TDG, which had the human acting directly and the agent acting directly, with no gate between them.

**Operational test:** Can the approval requirement be turned off for a condition without editing the game logic? If yes, the build is the controlled one.

**In my project:** Two source trees, one `vmiloserdov` and one `aduyinuo`, five overlapping files each, kept separate on purpose. The working convention is written into the merge README: keep each variant independently runnable, and put cross-variant comparison work in the `integration` folder rather than mixing files across source folders.

### Interactive Defense Game, IDG

**Definition:** The single-defender web game that came first. A Django application presenting the network as a table of hosts with IP address, name, subnet, last detected activity and compromised level, four action buttons, and two score readouts.

**Distinguishes it from:** TDG. IDG has one defender and no teammate, so it has no approval, no shared account, and no partner outcomes to learn from. It exists to put a person and a model through the identical scenario.

**Operational test:** Count the defenders. One is IDG.

**In my project:** The AAAI paper describes the interface and the two readouts. Last round gives the points lost from the action just taken. Total loss gives the running account for the episode.

## episode

**Definition:** One run of the scenario from a reset network to a fixed step count, and the unit over which the agent’s credit is finally assigned.

**Distinguishes it from:** A step, which is one turn of attacker action followed by defender action. Learning inside an episode moves values. Learning at the end of an episode rewrites them.

**Operational test:** Ask what resets. If the host statuses go back to their initial values and the loss account goes to zero, an episode ended.

**In my project:** Twenty-five steps. The simulation experiment ran 40 IBL defenders for 2000 episodes each. The human experiment gave 120 participants seven episodes.

## 1.2 Approval

### control dependency

*“Approved, the action executes. Left alone, it expires and the moment passes.”*

website, Team Defense Game, Approval

**Definition:** A directed requirement that one agent obtain another’s authorization before a listed action executes. The requirement is attached to named actions, not to the agent as a whole.

**Distinguishes it from:** An advisory recommendation, which the human may ignore with no effect on whether the action happens. Under a control dependency the default is that nothing happens. Silence is a denial, not a deferral.

**Operational test:** Take the approver out of the room for one turn. If the proposed action still executes, there was no control dependency, only a notification.

**In my project:** Implemented as `ApprovalAction`, a wrapper around the `Action` class that holds the approving agent. `DAG.add_control_edge` takes a start agent, an end agent, and the list of action names that need signing off. The open item on the pilot build was to make sure that only the person on the control edge can approve, and that one proposal cannot be approved twice.

### pending strip

**Definition:** The place in the interface where a proposed action waits between being proposed and being resolved. It is the visible form of the approval queue.

**Distinguishes it from:** The chat channel, which carries what the agent says about an action. The strip carries the action itself, in a state where it has not happened yet and has not been refused either.

**Operational test:** Is the item in it an action that will execute on a click, or a sentence about an action? Only the first belongs in the strip.

**In my project:** Her page puts it at the centre of the approval section. The pilot study’s open items track it as the action approval page, and record the interface fault worth remembering: when the action changes on the approval page it has to change on the main page too.

## authority

*“This is where authority actually lives. Not in a policy document, but in whether the action fires.”*

website, Team Defense Game, Approval

**Definition:** The disposition of who may cause what to happen, read off the graph rather than off a role description.

**Distinguishes it from:** Autonomy, which is about how much the agent decides on its own, and trust, which is about how the human treats what the agent says. Authority is settled before either of those is measured, and it is settled by the edge set.

**Operational test:** Name an action and ask whose signature makes it execute. If the answer needs a paragraph of policy rather than an edge, the authority is not in the game.

**In my project:** The chapter states the same point structurally: an action on a control edge can be approved only by the teammate on the other end of that edge, no matter how sensible it looks to anyone else.

## timeout policy

**Definition:** What happens to a proposal nobody answers. Two settings are available, auto-approve and auto-deny, and the setting is a condition rather than a default.

**Distinguishes it from:** The escalation chain, which decides who is asked next. The timeout decides what happens when nobody is asked next.

**Operational test:** Let a proposal sit for the full window and see what the network does. Whatever happens is the policy.

**In my project:** TDG runs the deny setting and describes it as expiry. The chapter lists both settings as configurable, together with escalation chains, risk tier gating, batch approvals and per-target scoping. Choosing between them is the throughput against safety comparison stated in the CHART modality section.

## 1.3 The shared pool

### pool dependency

*“Both defenders draw from the same budget. Every action either of them takes leaves less for the other, whether or not they coordinated.”*

website, Team Defense Game, The shared pool

**Definition:** On this page, a single account both defenders spend from, so that the two are joined even when they are acting on unrelated hosts.

**Distinguishes it from:** Two separate budgets of equal size, which produce the same per-action cost and none of the interference. The dependency is in the sharing, not in the pricing.

**Operational test:** Have one defender spend and ask whether the other’s remaining options changed. If they did not, the budget was not shared.

**In my project:** Her page states the consequence in the direction that matters: the human spending on one host narrows what the agent can do about another. That is the coordination cost being made visible without anyone being told to coordinate.

#### Pool names two different mechanisms

The website, Team Defense Game: a shared budget, where one defender’s spending subtracts from what the other can afford, and no joint action is triggered by anything.

The website, CHART, and the book chapter Table 1: contributions must reach a threshold before a composite action fires, with contribution weights, a sliding window, k-of-n quorum and a reset policy. Pool there is a trigger, not an account.

The pilot code, `blueAgent/00api.py: add_pooling_node` takes a `pool_threshold` that is a list of required (agent, action, target) tuples, so the condition is a required set being matched rather than a count being reached. Three locators, three mechanisms, one word.

The two senses are both defensible and the game uses only the first. A condition description that says pool without saying which one is ambiguous, and this is the term most likely to be read wrong by someone who learned it from the chapter.

### the loss account

**Definition:** The running total of defense points the blue side has given up in the current episode, decremented by both action costs and network state costs.

**Distinguishes it from:** The attacker’s reward, which is scored separately and does not mirror it. The two are not a zero sum: an attacker gains 10 for impact on the operational server while the defenders lose 10 for the same event, but a defender restore costs the defenders 1 and gains the attacker nothing.

**Operational test:** Take an action that changes no host status, such as monitor. If the account still moves, the cost was in the action. If it does not, the cost was in the state.

**In my project:** From the reward function in `blueAgent/00Env.py`: restore costs 1.0, misinform and shield cost 0.5 each, monitor and remove cost nothing to issue. Standing state costs are 0.1 for an exploited user host, 1.0 for an exploited server or a critical host, and 10.0 for impact on `Op_Server0`. Attacker rewards over the same events are 0.1 for discovering remote systems, 0.2 for a scan, 0.6 for an exploit, 1 for privilege escalation and 10 for impact.

## 1.4 The agent’s memory

### instance

*“Each entry is a situation paired with an action, and a value blended from the instances it has stored.”*

website, Team Defense Game, Learning from the partner

**Definition:** A memory unit holding three elements: a situation, described by a set of attributes; the action taken in it; and the outcome experienced.

**Distinguishes it from:** A transition in a replay buffer, which is also a situation, action and outcome triple. The difference is what is done with it. A replay buffer supplies gradient steps to a parametric policy. Instances are retrieved and averaged at decision time, and there is no policy separate from the memory.

**Operational test:** Delete one entry and ask whether one specific past experience is gone. In an instance memory it is. In a trained network it is not locatable.

**In my project:** The AAAI paper gives the slot structure for this task: two slots per host, the observed activity and the known compromised status, in a fixed order that encodes host identity, plus a step index slot. The decision is a host and a tool, written `cmd host`.

### activation

**Definition:** A per-instance quantity standing for how available that memory is at the moment of retrieval, driven by how often and how recently it was used, with a decay parameter and a noise parameter.

**Distinguishes it from:** The value stored in the instance. Activation decides whether the instance is retrieved at all. The stored outcome decides what it contributes once it is.

**Operational test:** Present the same situation twice in a row and see whether the second choice is more stable than the first. Recency entering the retrieval is activation at work.

**In my project:** Named in the AAAI paper with its two parameters, and cited to the ACT-R account of memory availability rather than derived there.

### blended value

**Definition:** The expected value of an option, formed as a probability-weighted average of the outcomes of all instances stored for that option, with the probabilities coming from activation. The choice rule is to take the option with the maximum blended value.

**Distinguishes it from:** A Q-value, which is one number per state-action pair updated in place. A blended value is recomputed from the surviving instances every time it is needed, so it can move without any learning step having occurred.

**Operational test:** Ask whether adding a new instance for an option can change that option's value without any parameter being updated. Under blending it can.

**In my project:** Her page describes the same quantity from the interface side, as a value blended from the instances the agent has stored, shown on the right of the screen while the episode runs.

### delayed feedback and credit assignment

*“At the end of the episode the whole thing re-settles at once: every instance from that episode is re-scored against how the episode actually went. Credit isn't assigned move by move. It's assigned in retrospect, to everything the team did.”*

website, Team Defense Game, Learning from the partner

**Definition:** The mechanism that writes outcomes into instances whose result was not known when they were stored, by distributing the episode's realized result back over the episode's decisions once it is available.

**Distinguishes it from:** Bootstrapping, which propagates a value estimate one step at a time

during the episode and needs no terminal rewrite. Credit assignment here happens once, at the end, over the whole episode.

**Operational test:** Interrupt an episode before it terminates and ask whether the memory has been updated at all. The within-episode shifts will have happened. The re-scoring will not.

**In my project:** Two things update, and confusing them is the error worth avoiding. The values shift as the episode runs, including from outcomes the human produced. The re-scoring at the end is a second, separate event that touches every instance from that episode.

## learning from the partner

*“The agent never chose those actions, but it lives with their consequences, so it learns from them.”*

website, Team Defense Game, Learning from the partner

**Definition:** The property that outcomes produced by the human teammate enter the agent’s memory, because the agent’s situation slots record network state rather than agent action history, and the network state carries what the human did.

**Distinguishes it from:** Imitation learning, which needs the partner’s action as a label and trains toward reproducing it. Here the human’s action is never recorded as a target. Only its effect on the board is, and the agent’s own action slot holds whatever the agent itself chose.

**Operational test:** Ask whether the agent could reconstruct which action the human took. If it cannot, and its values still moved, this is the channel.

**In my project:** This is the third channel, and the only one of the three that runs in one direction. Approval flows human to agent. The account is shared both ways. This one flows agent-ward only.

## Cognitive agent

**Definition:** The teammate whose actions come from episodic memory, as against a teammate whose actions come from fixed heuristics or from a uniform draw.

**Distinguishes it from:** The Heuristic agent, which is competent and steady, and the Random agent, which is neither. The published comparison is that the Cognitive agent is the most effective partner of the three, that it is more adaptive to individual play styles, and that it is also less predictable than the Heuristic one.

**Operational test:** Change the human’s play style across two sessions and read the agent’s behaviour. Adaptivity to the partner separates the first from the second. Consistency separates the second from the first in the other direction.

**In my project:** In the merged Controlled-TDG variants the blue setting names are random, collaboration and individual, and the individual branch loads a stored model trained for 500 episodes from IBL-individual/500.obj.

## 1.5 The board

### the defender action set

**Definition:** The commands the blue side can issue against one host per turn.

**Distinguishes it from:** The attacker's set, which is a progression rather than a menu: discover remote systems, scan, exploit, escalate privilege, impact. The defender's actions do not chain.

**Operational test:** Ask whether an action needs a prior action to have succeeded. If it does, it is on the red list.

**In my project:** Monitor observes and costs nothing. Remove clears attacker access short of privileged access and costs nothing to issue, except where the target is the host the benign user is on. Restore returns a host to a clean state and costs 1. Misinform plants a false picture for a fixed number of turns and costs 0.5. Shield costs 0.5 once when used.

#### Which four actions, and whether there are five

The Journal of Cybersecurity paper and the AAAI paper both give the defender four buttons: Monitor, Analyze, Remove, Restore.

Cyborg.py in the merged workspace gives four command values, and they are not the same four: 1 Monitor, 2 Remove, 3 Restore, 4 Misinform. Analyze is gone and deception has taken its slot.

blueAgent/00Env.py prices five: restore, misinform, shield, monitor, remove.

The set grew as the game moved from one defender to two, and a written condition that says the standard four actions no longer names a fixed set. Whichever paper is being cited, the action list has to be given rather than referred to.

### host status and activity

**Definition:** The two things known about each machine at each step. Activity is what was observed on it this turn. Status is how far the attacker has got on it.

**Distinguishes it from:** Each other, and the difference is what monitoring buys. Activity is noisy and immediate. Status is what the defender infers, and it persists.

**Operational test:** Clear all activity at the start of a turn, which the game does, and see what survives. What survives is status.

**In my project:** The status values in the code are Undetected, Scanned, Detected, Exploited, PrivEscd and Trapped. The pair per host is also the slot structure of the agent's instances, which is why the agent and the person are looking at the same thing.

### Beeline and Meander

**Definition:** The two scripted attackers. Beeline knows the network and drives straight at the operational server. Meander does not, and works through the hosts it can reach, taking privileged access as it goes.

**Distinguishes it from:** An adaptive attacker, which neither of them is. Both are deterministic given the board, which is what makes them usable as a fixed difficulty rather than as an opponent to be modelled.

**Operational test:** Play the same defense twice. If the attack path is the same, the attacker is one of these two.

**In my project:** Human defenders lose more against Beeline than against Meander, and improve more against Beeline over episodes. The IBL model shows the same two patterns, which is the result the Turing-like comparison rests on.



## Green agent

**Definition:** The simulated benign user, generating ordinary activity on hosts that the defender has to tell apart from an attack.

**Distinguishes it from:** Background noise added to observations. The green agent acts on hosts, so its activity is real network activity with a benign cause, and defender actions can hurt it.

**Operational test:** Take a defense action on the host the green agent is using and see whether the account moves. If it does, the user was modelled, not simulated as noise.

**In my project:** Present or absent as a condition. Removing an attacker from the green agent's host costs 0.5 where the same action elsewhere is free, which prices the cost of defending over a working user.

## 1.6 The evaluation

### human-autonomy team

**Definition:** A team in which at least one member is a person and at least one is an autonomous system that acts rather than advises, evaluated as a unit.

**Distinguishes it from:** A human using a tool. The test is whether the autonomous member has actions of its own that change the state, and whether the outcome measured belongs to the pair rather than to the person.

**Operational test:** Score the episode. If the number that matters is the person's score and the agent's contribution appears only as help, this is a tool study.

**In my project:** The abbreviation HAT is the one the CHB:AI paper uses throughout, and the study is the comparison of three teammate designs inside it.

### team process metrics

**Definition:** Measures of how the pair worked rather than of what they scored: how much the human had to do, how far the agent moved toward the human's style, how steady the agent was across situations.

**Distinguishes it from:** Team performance, which is the loss account at the end of the episode. The published result is that the two come apart. The most effective partner is also the least predictable one.

**Operational test:** Hold the score fixed across two conditions and ask whether anything differs. Whatever differs is a process measure.

**In my project:** The reported findings are adaptivity to individual play styles, inconsistency and lower predictability for the cognitive teammate, less human effort required by the two competent teammates, and higher trust and cooperation ratings for competent teammates than for the random one.

### over-reliance

**Definition:** The failure in which a competent partner is accepted more than its accuracy warrants, so that the pair's outcome tracks the partner rather than the pair.

**Distinguishes it from:** Appropriate reliance, which also looks like agreement. The two are only separable on the cases where the partner is wrong, so a measure that does not condition on partner correctness cannot detect this.

**Operational test:** Split the trials by whether the agent was right. If acceptance is flat across the split, the human was not discriminating.

**In my project:** Reported as a cost of competence: the competent agents require less human effort but might cause over-reliance. This is the same warning FriendOrFoe is built to test, arriving from the teammate side rather than from the explanation side.

## Turing-like experiment

**Definition:** A comparison in which a model and a person perform the identical task under the identical conditions, and the model is assessed by how far its behaviour and its learning curve resemble the person's.

**Distinguishes it from:** A benchmark, which asks only whether the model scores well. Resemblance can hold while performance differs, and the AAI result is exactly that shape: the same two patterns across attackers, at different efficiency.

**Operational test:** Ask what would count as the model doing too well. If nothing would, the comparison is a benchmark.

**In my project:** The two things held identical are the adapted CAGE scenario and the step count. The person plays it in IDG and the model plays it in CybORG.

## 2 Working

### DAG

The directed acyclic graph over teammates and pool nodes whose edges carry the dependency type and its parameters. In the pilot API it is a `networkx.MultiDiGraph`, which permits more than one edge between the same pair, so two dependency types can hold between the same two teammates at once.

### ApprovalAction and PoolActionWrapper

The two wrappers around the `Action` class that implement the two dependencies TDG uses. The first holds the approving agent and gates execution on it. The second diverts the call into a buffer instead of executing it. `TeamConfigurator` reads the graph and installs whichever wrapper each edge calls for, which is what makes the structure a configuration rather than a code change.

### trigger edge

The sixth edge type in the API specification, absent from the five named in the chapter. It carries the composite action out of a pool node into the environment, and it is where a control requirement gets attached to the pooled action itself rather than to any contributor's action.

### lateral movement

The attacker's progression from a compromised host into a subnet it could not previously see. In

the code it is not an attacker action but a consequence rule: privileged access on one named host makes the next subnet's hosts visible.

### **privilege escalation**

The step that turns a foothold into administrative control of a host. It is the status that standing costs are charged on, and the status that Remove cannot reverse, which is why the expensive Restore exists.

### **impact**

The attacker's terminal action on the operational server, worth 10 to the attacker and 10 against the defenders in one step. It is the only event in the game whose single-step magnitude is larger than a whole episode of ordinary attrition.

### **CybORG and CAGE**

The simulation gym and the challenge scenario the game is built on. Both were made to train and benchmark reinforcement learning defenders, with the human outside the loop as an evaluator. Everything TDG adds is on the human side of that line.

## **3 Peripheral**

### **Input, Process, Outcome**

The cycle the trace is organized along in CHART, and the reason the TDG traces are worth keeping in the same shape: setup as input, approvals and consultations as process, results and trust and workload as outcome, feeding forward.

### **alert fatigue**

The operational condition that motivates the whole line of work, where false alarm rates high enough to dominate the alert stream degrade the analyst's ability to escalate the true ones. TDG does not model it. FriendOrFoe does.

### **preference data**

The secondary use of the trace. Approvals mark acceptable behaviour, denials mark boundaries, and a modified proposal names the preferred alternative. The claim is about what the logs already contain, not about a training run that has been done.

## 4 The clashes, listed

| The disagreement                                                    | The entry it sits in                                     |
|---------------------------------------------------------------------|----------------------------------------------------------|
| Pool as a shared account, as a quorum trigger, or as a required set | pool dependency                                          |
| Four actions, and which four                                        | the defender action set                                  |
| Control on the platform, leader in the study                        | control dependency, and <code>hac_friendorfoe.tex</code> |
| Values shifting within an episode against re-scoring at its end     | delayed feedback and credit assignment                   |

The first and the third are the same kind of problem: a word that survived a move between two documents while its mechanism changed underneath it. The second is a version history rather than a disagreement, and is fixed by naming the action list wherever it is cited.

## 5 What crosses into the shared lexicon

Terms this project uses that the whole Human-AI Complementarity cluster uses. Their cluster sense goes in the shared file, and the entries above carry only what TDG fixes about them: complementarity, interdependence, taskwork, teamwork modality, authority, trust calibration, over-reliance, approval, explanation, dependency graph, and the five dependency types.

Two of those already differ across the cluster and stay in both files. Pool means a shared account here and a quorum trigger in CHART. Control keeps its own name here and is called leader in FriendOrFoe, where the mechanism is the same and the instantiation is a human holding the final decision rather than a supervisor holding a veto.

One term is fixed here and nowhere else. Learning from the partner names a channel that only exists once the teammate has a memory and the memory reads the board rather than the teammate. It has no counterpart in CHART, whose traces record what the human did but whose agents are not required to learn from it during the session.

## 6 Sources

- Every quoted sentence is from the Team Defense Game page of the website. The locator on each entry says which section.
- The approval and pool implementations, the graph API and the six edge types are from `blueAgent/00api.py` and `apiSpec.md`, and the two wrappers are named in the pilot study README.
- The costs, the status values, the action set and the attacker rewards are read from the two `blueAgent` files `00Env.py` and `00Cyborg.py`, and from `Cyborg.py` in the merged workspace.
- The interface, the episode and step counts, the instance slot structure and the account of activation, blending and delayed feedback are from the AAAI Symposium Series paper, read from the published PDF. The four-button action set and the two attackers are from it and from the Journal of Cybersecurity paper.

- The three teammate designs, the process metric findings and the over-reliance result are from the CHB:AI paper as the journal publishes its abstract.
- The dependency table, the timeout settings and the graph-walking argument are from the CHART book chapter manuscript, and belong properly to `hac_chart.tex`. They appear here only where TDG uses them.
- The three tiers, the three fluency levels and the four fields are from the Tech Fluency page. The reason a term whose two uses disagree belongs inside its entry is in `lexicon_method.tex`, in this folder.